

OFQ-LLM: Outlier-Flexing Quantization for Efficient Low-Bit Large Language Model Acceleration

Gang Wang[✉], Siqi Cai[✉], Wenjie Li[✉], Dongxu Lyu[✉], and Guanghui He[✉], *Member, IEEE*

Abstract—Large Language Models (LLMs) have achieved significant success in various Natural Language Processing (NLP) tasks, becoming essential to modern intelligent computing. Their large memory footprint and high computational cost hinder efficient deployment. Post-Training Quantization (PTQ) is a promising technique to alleviate this issue and accelerate LLM inference. However, the presence of outliers impedes the advancement of LLM quantization to lower bit levels. In this paper, we introduce OFQ-LLM, an algorithm-hardware co-design solution that adopts outlier-flexing quantization to efficiently accelerate LLM at low-bit levels. The key insight of OFQ-LLM is that normal data can be efficiently quantized in a slightly reduced data encoding space, while the rest encoding space can be used for flexible outlier values. During quantization, we use rescale-based clipping (RBC) to optimize accuracy for normal data and group outlier clustering (GOC) to flexibly represent outlier values. At the hardware level, we introduce a memory-aligned outlier-flexing encoding scheme to encode activations and weights in LLMs at a low bit level. The outlier-normal mixed hardware architecture is devised to leverage the encoding scheme and accelerate LLMs with high speed and high energy efficiency. Our experiments show that OFQ-LLM achieves better accuracy compared to state-of-the-art (SOTA) low-bit LLM PTQ works. OFQ-LLM-based accelerator surpasses the SOTA outlier-aware accelerators by up to 2.69× core energy efficiency, up to 3.83× speed up and 2.44× energy reduction in LLM prefilling phase, and up to 2.01× speed up and 2.88× energy reduction in LLM decoding phase, with superior accuracy.

Index Terms—Large language models, post-training quantization, algorithm-hardware co-design, outlier, efficient acceleration.

I. INTRODUCTION

RECENTLY Large Language Models (LLMs) have achieved remarkable success across a wide range of Natural Language Processing (NLP) tasks [1], [2] and are garnering widespread attention from both academia and industry. With the continued scaling of model size [3], LLMs demonstrate superior performance across many downstream tasks. Numerous LLMs including GPT series (GPT-3 [4],

GPT-4 [5]), LLaMA-series (LLaMA2 [6], LLaMA3), and OPT [7] are widely used for both academic research and commercial purposes. Furthermore, starting from the explosive popularity of ChatGPT, LLMs have rapidly become an indispensable part of our work routines and daily lives, handling diverse tasks such as summarization, reasoning, and code completion. LLMs are widely regarded as paving the way towards future Artificial General Intelligence (AGI) for humanity [8].

However, the remarkable performance of LLMs comes with billions of model parameters and significant computation complexity. For example, GPT3 [4], a notable LLM, has 175 billion model parameters and requires 350GB of memory in FP16 format, equivalent to at least five NVIDIA A100-80G GPUs for single model deployment. This substantial burden on memory and computational resources greatly hinders the practical deployment of LLMs in real-world applications.

Quantization is one of the most popular techniques to reduce the memory storage of LLMs, compressing them by converting activations and weights to low-bit integers, thereby significantly reducing memory storage requirements. One of the quantization techniques, quantization-aware training (QAT) achieves higher accuracy but incurs significant training costs, making it less feasible for LLMs with billions of parameters. In contrast, post-training quantization (PTQ) can compress the pre-trained model in one shot without requiring additional training, making it an appealing and practical method for LLMs. Most PTQ for LLMs can be divided into two categories: weight-only quantization [9], [10], [11], [12] and weight-activation quantization. Weight-only quantization focuses solely on compressing weight into integers while keeping activations in floating-point format. This quantization scheme reduces the overhead of storing model parameters, but requiring a weight dequantization process and costly floating-point arithmetic. In contrast, weight-activation quantization [13], [14], [15], [16] convert both weights and activations into low-bit integers, making it compatible with hardware-friendly integer arithmetic units, which leads to a lower memory footprint, reduced computational complexity, and lower power consumption. In this paper, we focus on low-bit weight-activation PTQ in W4A4 (4-bit for both weights and activations) and W4A8 with algorithm-hardware co-optimization to enhance both model accuracy and the energy efficiency.

Received 28 August 2024; revised 30 December 2024; accepted 27 February 2025. Date of publication 13 March 2025; date of current version 30 July 2025. This work was supported by the National Science Foundation of China under Grant 92464302. This article was recommended by Associate Editor X. Zeng. (Corresponding author: Guanghui He.)

Gang Wang, Siqi Cai, Dongxu Lyu, and Guanghui He are with the State Key Laboratory of MicroNano Engineering Science, School of Integrated Circuits, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: wang-gangjin@sjtu.edu.cn; caisiqui-2020@sjtu.edu.cn; lvdongxu@sjtu.edu.cn; guanghui.he@sjtu.edu.cn).

Wenjie Li is with the School of Computer Science, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: wenjieli@sjtu.edu.cn).

Digital Object Identifier 10.1109/TCSI.2025.3547732

Weight-activation PTQ for LLMs can not only reduce the storage requirements but also attain faster inference times. As a result, significant research efforts have been dedicated to lowering the quantization precision to low bits [11], [16]. However, achieving high quantization accuracy in low-bit LLM quantization remains challenging due to the presence of outliers [17]. Outliers are a tiny fraction ($< 0.1\%$) of activations and weights in LLMs with exceptionally high magnitudes but have a significant impact on model accuracy [10]. Clipping or removing these outliers will result in a noticeable degradation in model accuracy [10], [15]. To tackle this challenge, SmoothQuant [13] attempts to shift the outlier quantization challenge from activations to weights through a per-channel scaling-based mathematically equivalent transformation, but suffers from significant accuracy loss in low-bit quantization, such as W4A8. Atom [18] and QUIK [19] maintain the outliers in int8 and FP16 separately, and accelerate the computation through an int-float hybrid dataflow, which leads to unbalanced workload and limited efficiency gains. RPTQ [14] pushes the bit width limit to W4A4 by using reorder-based channel clustering quantization, but still lack further outlier optimizations. Therefore, existing outlier-aware quantization schemes either face significant accuracy loss at low-bit levels or handle outliers in a hardware-unfriendly manner, leading to an inefficient computational paradigm.

A number of algorithm-hardware co-design approaches have been proposed to deal with the outliers in LLMs [20], [21], [22], [23]. GOBO [22] and OLAcel [23] both utilize a sparse outlier encoding format to separately store the value and coordinate of each outlier. However, these outlier-aware solutions introduce complex dense-sparse hardware overhead. OliVe [21] employs outlier-victim patterns to represent significant outlier values within original memory-aligned tensors, but this memory-aligned pattern comes at the cost of non-negligible accuracy loss. As a result, it remains challenging to design low-bit LLM PTQ that can not only achieve better model accuracy but also improve energy efficiency with hardware-friendly architecture support.

To tackle these challenges, we propose a novel outlier-aware quantization scheme, OFQ-LLM, which adopts Outlier-Flexing Quantization to efficiently accelerate LLM at low-bit levels. The key insight of OFQ-LLM is to quantize the normal data in a slightly reduced data range, e.g. $[-6, 7]$ for 4-bit quantization, while utilizing the left data encoding space (-7) to flexibly represent outlier values with significant magnitude. OFQ-LLM divides the normal and outlier data into corresponding data ranges based on the characteristics of the data distribution. As a result, OFQ-LLM can not only optimize the quantization of both normal and outlier data simultaneously but also introduces an aligned low-bit encoding scheme, which is hardware-friendly and promising for hardware acceleration.

OFQ-LLM optimizes the accuracy and efficiency of LLMs in an algorithm-hardware co-design manner and can be seamlessly integrated into weight-activation PTQs and LLM accelerators to fully leverage our advantages. Our main contributions are listed as follows:

- This work proposes **Rescale-Based Clipping (RBC)** to flexibly divide the normal and outlier data into corresponding data ranges. RBC adjusts the quantization scaling factor and clips the normal data into the reduced data range, which not only optimizes the normal data quantization but also preserves a small encoding space for outlier values.
- **Group Outlier Clustering (GOC)** is introduced to efficiently represent outliers. We divide the data into groups and aggregate in-group outliers into clusters using a heuristic K-means algorithm, where the number of clusters is equal to the remaining encoding space. The mean of each cluster is then used as the representative outlier for a specific group, allowing for low-bit encoding while maintaining significant outlier flexibility.
- In the hardware perspective, an outlier-flexing encoding scheme is proposed to fully capitalize the opportunities introduced by RBC and GOC. This encoding scheme leverages a small GOC LUT to store the group-based outlier encoding values and corresponding outlier values. Additionally, an outlier-normal mixed architecture is devised to efficiently translate data into normal values and outlier values and accelerate the corresponding workloads with high speed and high efficiency.
- Experiments on various LLM models are conducted and reveal that our quantization scheme achieves better performance than the state-of-the-art (SOTA) low-bit LLM PTQ methods. In addition, the OFQ-LLM-based accelerators achieve up to $2.69\times$ core energy efficiency, up to $3.83\times$ speed up and $2.44\times$ energy reduction in LLM prefilling phase, and up to $2.01\times$ speed up and $2.88\times$ energy reduction in LLM decoding phase compared to the SOTA outlier-aware accelerators, while maintaining superior accuracy.

The rest of the paper is organized as follows. Section II presents the background and preliminaries for LLMs. In Section III, we offer an in-depth description of our proposed quantization scheme. Subsequently, Section IV elaborates on the details of OFQ-LLM-based hardware architectures. Experimental results and comparisons are given in Section V to demonstrate the superiority of our work. Finally, Section VI draws the conclusion.

II. BACKGROUND AND RELATED WORKS

A. Large Language Models

The Transformer-based language model was initially designed as a unified architecture for natural language processing (NLP) and achieved tremendous success [24]. Since the introduction of GPT [4], the large pre-trained decoder-only Transformer has become the most commonly used architecture for LLMs. As shown in Fig. 1, the main architecture of an LLM consists of multiple stacked Transformer decoder layers. Each Transformer decoder layer receives the token embedding from the previous layer, passes the embedding through each submodule, and generates the token embedding for the next layer. A Transformer layer comprises two layer

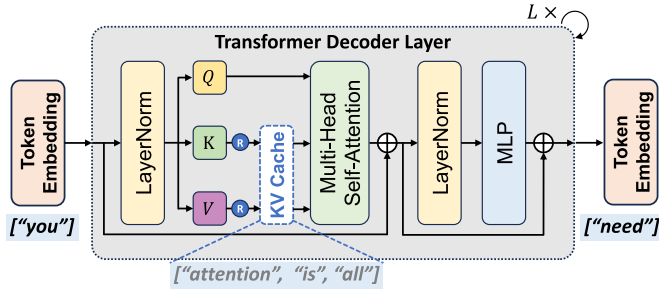


Fig. 1. Illustrating the large language model (LLM) architecture.

normalization (LayerNorm) modules, an attention module and a multi-layer perceptron (MLP) module. The inference of LLMs can be divided into two phases: the prefilling phase and the decoding phase [20]. In the prefilling phase, the LLM processes the sequence of tokens in the user prompt using general matrix-matrix multiplication (GEMM) to generate the associated KV cache. In the decoding phase, the LLM takes the most recently generated token as input with a sequence length of one, and computes the next token using general matrix-vector multiplication (GEMV). This decoding phase iterates in a self-regression manner to sequentially generate all response tokens. Inside the Transformer decoder layer, the token embedding first passed through a LayerNorm module, producing output tokens denoted as $X \in \mathbb{R}^{n \times D}$. The attention module then projects the token to query Q , key K and value V across h heads as:

$$Q = XW^Q, K = XW^K, V = XW^V, \quad (1)$$

where W^Q , W^K and W^V are the projection weights. In recently released LLMs such as LLaMA, and LLaMA2 [6], rotary position embedding (RoPE) [25] is applied after key and value to inject position information into the model. KV cache [6], [7] is used to store the keys and values previously computed in the attention. In the decoding phase, LLMs retrieve the previous keys and values, then concatenate them with the current key and value and perform multi-head self-attention (MHSA). The attention output A can be computed as:

$$\begin{aligned} A &= \text{MHSA}(Q, K, K_{\text{Cache}}, V, V_{\text{Cache}}) \\ &= \text{Softmax}\left(\frac{Q[K, K_{\text{Cache}}]^T}{\sqrt{d_k}}\right)[V, V_{\text{Cache}}], \end{aligned} \quad (2)$$

where K_{Cache} and V_{Cache} represent the keys and values fetched from KV cache. Finally, the MHSA concatenates the attention outputs from all heads and applies a linear projection to generate the final output O as:

$$O = [A_1, A_2, \dots, A_h]W^O, \quad (3)$$

where W^O is the output projection weights.

An MLP is used to transform token embeddings after MHSA with linear layers and an activation function module. A typical MLP can be defined as:

$$\text{MLP}(X) = W^2 \sigma(W^1 X), \quad (4)$$

where X denotes the token embeddings, W^1 and W^2 denotes the weights in linear layers, and $\sigma(\cdot)$ denotes the activation function. Unlike traditional Transformers, the activation functions vary among different LLMs. For example, OPT [7] adopt ReLU while LLaMA2 adopt SwiGLU [26] respectively.

B. Related Quantization Schemes

Quantization is a technique that compresses the original floating-point values with low-bit integers. Formally, a m bit quantization can be defined as:

$$q = \left\lceil \frac{x}{s} \right\rceil + z, \quad (5)$$

where q is the quantized m -bit integers, x is the original floating-point values, and $\lceil \cdot \rceil$ denotes the round function. The corresponding dequantization can be defined as:

$$\hat{x} = s(q - z) \approx x. \quad (6)$$

Furthermore, s is the scaling factor and z is the zero-point, both of which can be computed by the min and max values of x from the calibration dataset. Scaling factor and zero-point are determined as follows:

$$s = \frac{x_{\max} - x_{\min}}{b_{\max} - b_{\min}}, z = \left\lceil -\frac{x_{\min}}{s} \right\rceil, \quad (7)$$

$$b_{\max} = 2^{m-1} - 1, b_{\min} = -2^{m-1}, \quad (8)$$

where the quantized m -bit integer values fully covered the data range in $[b_{\min}, b_{\max}]$. In other words, $x_{\max}$ is mapped to $b_{\max}$, while $x_{\min}$ is mapped to $b_{\min}$.

Quantization of LLMs with low bitwidth is non-trivial due to the presence of outliers. Outliers are tiny fractions of data that typically have much higher magnitudes than normal values and significantly impact model accuracy. To address this issue, several outlier-aware quantization methods have been proposed [10], [13], [15]. Notably, LLM.int8() [10] retains outliers in floating-point, while SmoothQuant [13] and RepQuant [27] shift the outlier scales from activation to weights, easing the quantization difficulty of activations. Omniquant [28] and QUIK [19] take a different approach, using weight clipping to improve quantization by trimming the data distribution before rounding with an optimal clipping threshold. RPTQ [14] is the first reported work that pushes the bit width limit to W4A4. It applies reorder-based channel clustering to assign the outliers to the same group. These aforementioned quantization methods for LLMs either fail to achieve negligible accuracy loss at low-bit level or introduce hardware-unfriendly computation during inference, thereby limiting further efficiency enhancement.

Several quantization/hardware co-designs have been proposed to efficiently encode outlier values and accelerate Transformer inference [21], [22], [29]. GOBO [22] employs sparse coordinate encoding which extracts outliers using value-coordinate lists to present outliers using high precision, as shown in Fig.2(a). While this encoding method can maintain model accuracy, it unfortunately introduces unaligned memory access and additional sparse-related hardware overhead. In OliVe, outlier-victim encoding can store outliers with high precision in a memory-aligned fashion [21]. However, this

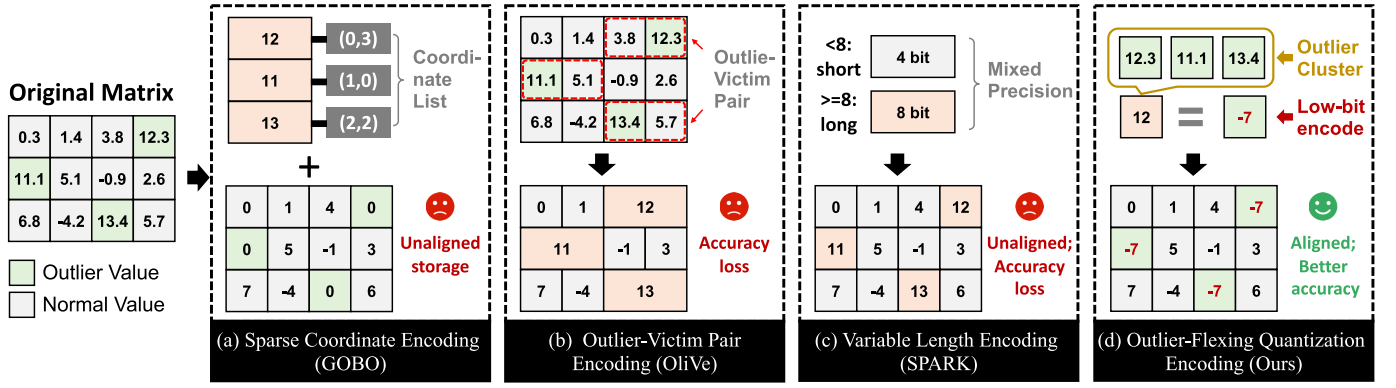


Fig. 2. Comparisons of outlier-aware encoding methods. (a) Sparse coordinate encoding that stores outlier and normal values separately. (b) Outlier-victim encoding that stores the outliers based on sacrificing adjacent normal values. (c) Variable length encoding that stores outlier and normal values with different lengths. (d) Our proposed outlier-flexing encoding that stores clustered outlier values and encodes them using the same low-bit width with normal values.

additional outlier storage is achieved by pruning adjacent values, which inevitably leads to accuracy loss. SPARK [29] represents values smaller than 8 with 4 bits while representing values larger than or equal to 8 with 8 bits. Such encoding not only stores values with variable lengths, leading to unaligned memory access, but also requires an inside 1-bit identifier inside each data to distinguish different lengths, which restrict the encoding ability in a given bitwidth and adversely affects the model accuracy.

To efficiently quantize and encode outliers with memory-aligned patterns while optimizing model accuracy, we propose a new outlier-aware quantization called “Outlier-flexing quantization” (OFQ), and its corresponding encoding is depicted in Fig. 2(d). Our OFQ clusters the outliers within a specific group into a few common values, and encodes them using the unused low-bit encoding space which in Fig. 2(d) is $[-7]$. This encoding efficiently stores outlier and normal values at low-bit levels. In addition, our encoding provides opportunities to further improve model accuracy by flexibly determining clustered outliers and normal encoding spaces.

III. PROPOSED OFQ-LLM QUANTIZATION SCHEME

In this section, we propose our OFQ-LLM quantization scheme. First, we give an overview and discuss the advantages of OFQ-LLM. Then, we describe RBC and GOC in detail.

A. OFQ-LLM Overview

The primary challenge in low-bit quantization for large language models (LLMs) is the presence of outliers. To illustrate this, we profiled and visualized the weight distribution in the LLaMA-7B model. Our analysis highlights the significant variance between outlier and normal values within LLMs. As depicted in Fig. 3, approximately 99% of the weights fall within a narrow data range of less than 0.06. In contrast, the 0.1% of outlier values exhibit significantly greater magnitudes than normal values. Furthermore, these outlier values show considerable variability and flexibility between each other. Consequently, naively quantizing LLMs while accounting for the extensive range of outliers results in significant accuracy degradation, especially at low-bit levels. As illustrated in Fig. 4(a), a number of encoding positions fall

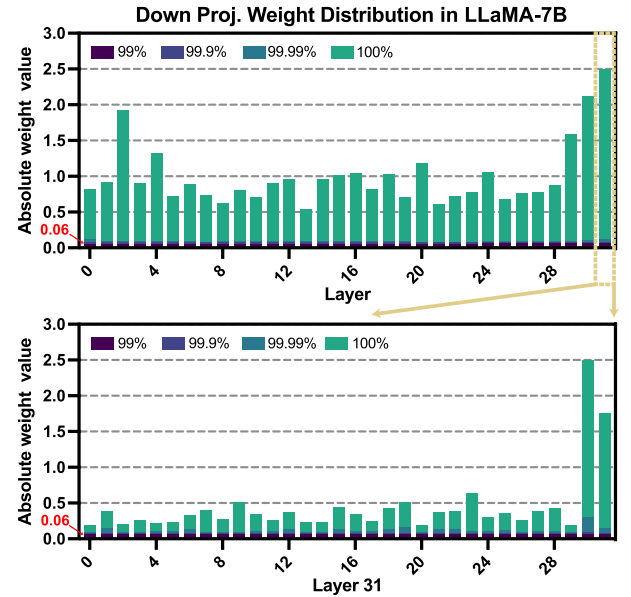


Fig. 3. The distribution of the absolute weights in down projection layers in LLaMA-7B. The outlier values exhibit great variance with each other and with normal values. 99% normal values share a similar and small data range.

into the near-outlier space during naive quantization, which covers only a tiny portion of the data. As such, the large range of outliers constricts the encoding space for the 99% of normal values, and the variability of outliers leads to insufficient coverage for these flexible values.

In this paper, we aim to tackle the aforementioned quantization challenges based on the observation from the distribution properties illustrated in Fig. 3. Specifically, we note that normal values occupy a narrow, similar range, while outlier values span a wider, more flexible range. To address this, we propose dividing the low-bit encoding space into two distinct sections: one with a slightly reduced encoding space for the majority of normal values, and another, smaller space reserved for the flexible outlier values. This division allows us to simultaneously optimize the quantization for both normal and outlier values.

To achieve this, we propose an outlier-flexing quantization scheme for large language models, named OFQ-LLM, which

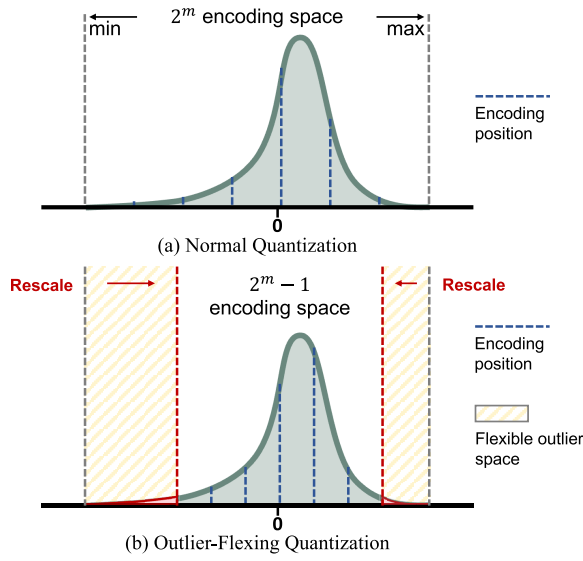


Fig. 4. Comparison of OFQ-LLM with normal quantization. For m -bit quantization, a) normal quantization uses the entire range from min to max to construct the 2^m encoding space. b) OFQ-LLM utilizes rescaled min and max values to construct a slightly reduced $2^m - 1$ encoding space for normal values, reserving the remaining encoding space for values in the flexible outlier space.

efficiently quantizes LLMs at low-bit levels. As is shown in Fig.4, unlike normal quantization that uses the entire encoding space based on min and max values, OFQ-LLM encodes normal values with a slightly reduced encoding space with rescaled min and max. The reduced encoding space covers $2^m - 1$ in this example. After rescaling, more encoding positions are available for normal data compared to naively quantization which help reduce the quantization error for normal data. Therefore, our OFQ-LLM quantization scheme quantizes normal values and outlier values separately. For normal values, we use a **Rescaling-Based Clipping (RBC)** method that adjusts the min and max values to be more concentrated around the normal values, followed by clipping and quantization. Even with a slightly reduced encoding space, OFQ-LLM can still guarantee more sufficient encoding positions and representation ability for normal values. For outlier values, which reside in the flexible outlier space, we introduce **Group Outlier Clustering (GOC)** to encode them with maximum flexibility into the left encoding space. Specifically, we employ a heuristic K-means algorithm to aggregate outliers in a group of activations or weights into several clusters. The number of clusters is determined by the remaining encoding space. The mean of each cluster is used to represent the outlier values in a specific group and is associated with the remaining low-bit encoding space. Thus, OFQ-LLM optimizes the quantization of both normal and outlier values through RBC and GOC. The following sections provide a detailed explanation of RBC and GOC.

B. Rescale-Based Clipping (RBC)

RBC follows the standard uniform quantization scheme but uses rescaled min and max along with a reduced encoding space. Assume the encoding space reduced by n , and the

rescale ratio is r , RBC can be expressed as:

$$q = \lceil \frac{x}{\tilde{s}} \rceil + z, x \in [rx_{min}, rx_{max}], \quad (9)$$

$$\begin{aligned} \tilde{s} &= \frac{rx_{max} - rx_{min}}{2^m - 1 - n}, r \in (0, 1), \\ &= r \frac{2^m - 1}{2^m - 1 - n} s = \Delta \cdot s, \end{aligned} \quad (10)$$

where Δ denotes the rescaling factor. Compared with normal quantization in Eq.5, RBC utilizes a new scaling factor $\tilde{s}$ to quantize the normal values which reside between rx_{min} and rx_{max} . These values are clipped into a $2^m - 1 - n$ encoding space, while the remaining values are categorized as outliers. According to Eq.10, the r can adjust the ratio between the rescaled scaling factor and the origin scaling factor. When $r = \frac{2^m - 1 - n}{2^m - 1}$, the rescaled scaling factor is exactly equal to the origin scaling factor. In this case, the min and max values are proportionally rescaled for the reduced encoding space, resulting in clipped normal values identical to those in the original quantization.

We further analyze the quantization error after RBC. Following RBC, the quantization for normal values can be expressed as:

$$\hat{x} = \Delta \cdot s \left(\lceil \frac{x}{\Delta \cdot s} \rceil \right). \quad (11)$$

As reported in [12], the expected error after applying the rounding operation remains consistent between different quantization schemes. The error from Eq.11 can be expressed as:

$$Err = \Delta \cdot s \cdot RoundErr, \quad (12)$$

From the Eq.12, we derive that the ratio of this error to the original quantization error is $\frac{Err'}{Err} = \Delta$. Consequently, if $\Delta < 1$, the relative quantization error is reduced for normal values post-RBC. For further illustration, when $\Delta < 1$, we have $r < \frac{2^m - 1 - n}{2^m - 1}$. This condition implies a smaller min and max is associated with the reduced encoding space, providing a larger encoding space for these values, as illustrated in Fig.4(b). In our proposed methodology, we consistently employ a narrower min-max range centered around these typical values. By doing so, we ensure a more concentrated quantization space, thereby improving the encoding granularity and reducing relative quantization error.

C. Group Outlier Clustering (GOC)

Besides RBC, we propose GOC for the quantization of outlier values. We divide weights and activation into uniform groups, such as a group size of 128, and use a heuristic K-means algorithm to aggregate outlier values within these groups. The GOC algorithm depicted in Algorithm 1 processes the quantized weights by outputting them with clustered outliers. The GOC method comprises three main steps. In the first step, we reshape the quantized weights into groups and extract outlier values, preparing them for group-wise clustering. Second, within each group, we eliminate clusters that contain too few outliers. As depicted in Fig.4(b), the outliers in the flexible outlier space are intuitively divided into two clusters: one consisting of elements larger than the rescaled max and another

Algorithm 1 Group Outlier Clustering

Input: Q : Quantized weights, G : group size, C : number of clusters, q_{max} : max encoding position from RBC, q_{min} : min encoding position from RBC

Output: O : Quantized weights with clustered outliers

```

1:  $Q_r = Q.\text{reshape}(-1, G)$  ▷ step 1
2: Outlier:  $Q_o = Q_r[Q_r > q_{max} \text{ or } Q_r < q_{min}]$ 
3: for group  $g \in [0:G]$  do ▷ step 2
4:    $Q_1 = Q_o[g][Q_r > q_{max}]$ 
5:    $Q_2 = Q_o[g][Q_r < q_{min}]$ 
6:    $d_1 = \text{sum}(|Q_1 - \text{clip}(Q_1, q_{max})|)$ 
7:    $d_2 = \text{sum}(|Q_2 - \text{clip}(Q_2, q_{min})|)$ 
8:   if  $d_1 < (d_1 + d_2)/2C$  or  $(C = 1 \text{ and } d_1 < d_2)$  then
9:      $Q_1 = 0$ 
10:  end if
11:  if  $d_2 < (d_1 + d_2)/2C$  or  $(C = 1 \text{ and } d_1 > d_2)$  then
12:     $Q_2 = 0$ 
13:  end if
14:   $Q_o[g] = Q_1 + Q_2$ 
15:  mean, index =  $\text{KMeans}(Q_o[g], C)$  ▷ step 3
16:   $Q_o[g][\text{index}] = \text{mean}$ 
17:   $Q_o[g] = Q_o[g] - \text{clip}(Q_o[g], q_{max}, q_{min})$ 
18: end for
19: Clipped Normal:  $Q_n = \text{clip}(Q_r, q_{max}, q_{min})$ 
20:  $O = Q_n + Q_o$ 
21:  $Q.\text{reshape}(Q.\text{shape})$ 

```

containing elements smaller than the rescaled min. This elimination occurs when the number of clusters is fewer than two or when the two aforementioned clusters are strongly unbalanced. We assess the balance of these two clusters by calculating each accumulated distance between the outliers and the clipped values, denoted as d_1 and d_2 in Algorithm 1. After eliminating the underpopulated cluster, we apply the K-Means algorithm to the remaining outliers, using the mean values of each cluster to represent outliers within each group. Meanwhile, outliers that are eliminated are replaced with their respective clipped min and max values to best preserve accuracy. Consequently, outliers are aggregated into representative groups, matched in number to the remaining encoding spaces after RBC. These representatives are then encoded into low-bit values using a lookup table, referred as *GOC LUT*. For instance, as depicted in Fig.2(d), outliers are aggregated into one cluster and utilize the left encoding space $[-7]$ to represent the actual outlier value (12). Both left encoding space values and corresponding outlier values are stored in the GOC LUT. We compare the proposed GOC with conventional K-means algorithm in Fig.5. Our GOC greatly outperform conventional K-means when group size is 1, since the outlier data points are clustered into a single group and the outlier with opposite sign would contaminate the conventional K-means results. In addition, the GOC achieves comparable results with K-means when group size is 2, which indicates that the GOC can effectively cluster the outlier values with more hardware friendly and lower complexity computation paradigm. When the cluster number grows to 3 and 4, the perplexity results increased, indicating that simply adding more clusters does not necessarily improve

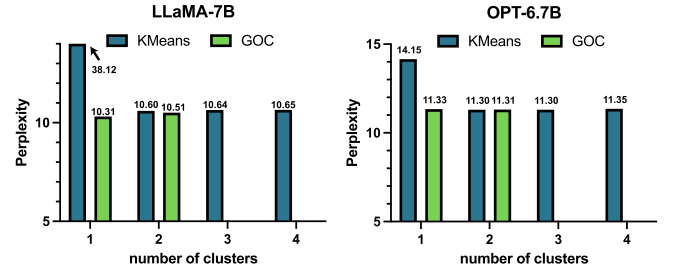


Fig. 5. Comparison of conventional K-means and GOC for LLaMA-7B and OPT-6.7B on WIKITEXT2.

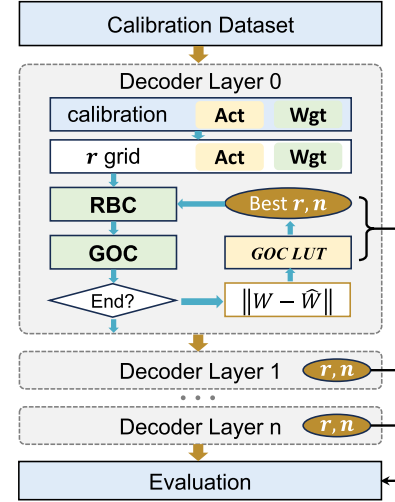


Fig. 6. Exploration for layer-wise OFQ-LLM configuration.

performance. Therefore, in this paper, the GOC LUT entry number or outlier space are set to 1 or 2 for outlier clusters.

D. Exploration for Layer-Wise OFQ-LLM Configuration

To determine the optimal rescaled ratio r and encoding space reduction n in RBC with minimized accuracy loss, we propose an exploration algorithm for layer-wise OFQ-LLM configuration. The optimization problem can be formally expressed as:

$$\arg \min_r \|W - \hat{W}\|, \quad (13)$$

$$\hat{W} = GOC(RBC(W, r, n)), \quad (14)$$

where W represents the quantized weights and $\hat{W}$ denotes the quantized weight after OFQ-LLM processing. As is shown in Fig.6, we conduct a layer-wise grid search to get the optimal r, n and corresponding GOC LUT. For each layer, we initially establish the original min and max for activations and weights through standard calibration. Subsequently, we generate a suitable r, n grid to search for the best r and n . During each specific grid search, RBC is applied to quantize the activations and weights using the identified r and n . The quantized values are then processed using GOC, as described in Algorithm 1, to aggregate clustered outlier values and store them in the GOC LUT along with the left encoding values. Finally, we utilize Eq.13 to assess the corresponding quantize error and select the best r and n . After the grid search and quantization are

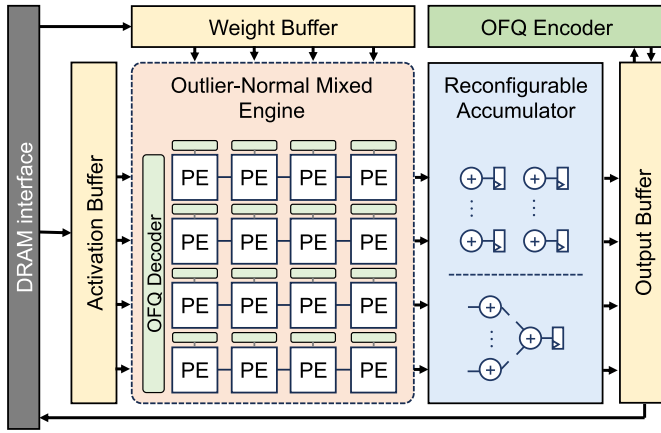


Fig. 7. Overview of OFQ-LLM architecture.

completed for all layers, we inference the quantized model on evaluation datasets.

It is noteworthy that the proposed OFQ-LLM quantization scheme is fully compatible and can be seamlessly integrated into the standard LLM PTQ scheme. By employing a quantization scheme that separately quantizes normal and outlier values, OFQ-LLM not only enhances the accuracy of the quantized model but also provides opportunities for efficient LLM acceleration.

IV. PROPOSED OFQ-LLM HARDWARE ARCHITECTURE

This section presents how to fully exploit OFQ-LLM with hardware acceleration to improve the LLM inference efficiency in both the prefilling phase and the decoding phase. LLM computational complexity is dominated by the FFN linear, Attention linear and Attention matmul layers. As is shown in [11], nonlinear operations only take less than 10% of total runtime on GPUs. As such, GEMM and GEMV are dominating LLM inference and make it arguent for hardware acceleration. This section introduces OFQ-LLM-based architecture tailored for GEMM and GEMV acceleration in LLMs. We first describe the overall architecture. Then the decoder and encoder for the aforementioned OFQ encoding. We further detail the outlier-normal mixed PE and reconfigurable accumulator. Finally, we elaborate on the GEMM and GEMV dataflow for LLMs in detail.

A. Architecture Overview

The overall OFQ-LLM-based architecture is shown in Fig.7. It comprises an activation buffer, a weight buffer, an output buffer, an outlier-normal mixed engine composed of OFQ decoders and PEs, a reconfigurable accumulator and an OFQ encoder. The activation buffer and weight buffer store OFQ-encoded activations and weights. The outlier-normal mixed engine processes the OFQ-encoded data with OFQ decoders, which convert the data with the left encoding values into clustered outlier values. After decoding, the normal and outlier data are processed in outlier-normal mixed PEs. These PEs gate the outlier-related logic when outliers are absent, and process outliers in an interleaved manner when they are

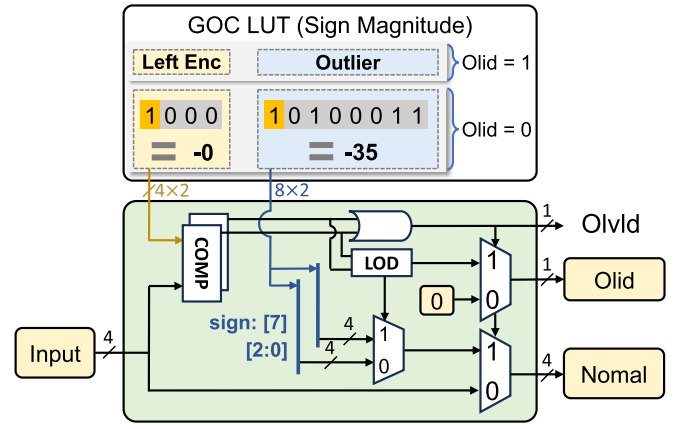


Fig. 8. The 4-bit OFQ decoder with GOC LUT. GOC LUT consists of outlier id (Olid), outlier value and left encoding value. Values are stored in sign-magnitude format. The input is compared with left encoding values and generates olid and normal values.

present. The outlier-normal mixed engine can be configured into GEMM mode and GEMV mode, and the outputs are transferred to the reconfigurable accumulator which can be configured to separate accumulation mode and adder tree accumulation mode. Finally, the output data pass through an OFQ encoder and output low-bit OFQ encoded values.

B. OFQ Decoder and Encoder

Our OFQ-LLM quantization scheme produces activations and weights with low-bit data and GOC lookup data, which can be efficiently encoded using the OFQ encoding scheme, as shown in Fig.2. To facilitate rapid OFQ decoding within our architecture, we propose an OFQ decoder characterized by simple logic and straightforward integration capabilities. Our 4-bit decoder, depicted in Fig.8, consists solely of multiplexers, comparators, an OR gate and a leading one detector (LOD). Alongside the decoder, a GOC LUT is utilized to store the left encoding value in 4-bit and the corresponding 8-bit outlier values. The data in our OFQ-LLM-based architecture are represented in a sign-magnitude format, which can be directly split into the most significant part (MSP) and least significant part (LSP).

During decoding, the decoder first compares the input value with the left encoding values. The results of this comparison are sent to the multiplexers to select the appropriate normal part and outlier id (Olid). If the input does not match a left encoding value, it is selected and output to the normal part, while the outlier id is assigned a value of 0. Conversely, if the input matches a left encoding value, an outlier valid (olvid) signal is set and the LOD generates the corresponding Olid and selects corresponding outlier values from the GOC LUT. The outlier is further split into a sign, the most significant part (MSP) from bits [6:3], and the least significant part (LSP) from bits [2:0]. The sign and LSP are concatenated to form the normal part, and the sign and MSP are concatenated to form the outlier part. As shown in Fig.8, our OFQ decoder capitalizes on the redundant encoding space in the sign-magnitude format (−0), effectively overcoming the inherent encoding weaknesses of this format.

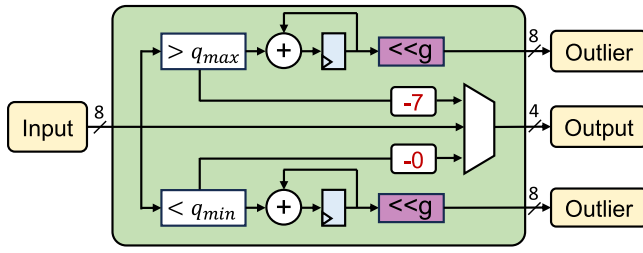


Fig. 9. Illustration of OFQ encoder. In this case, there are two left encoding spaces -0 and -7 .

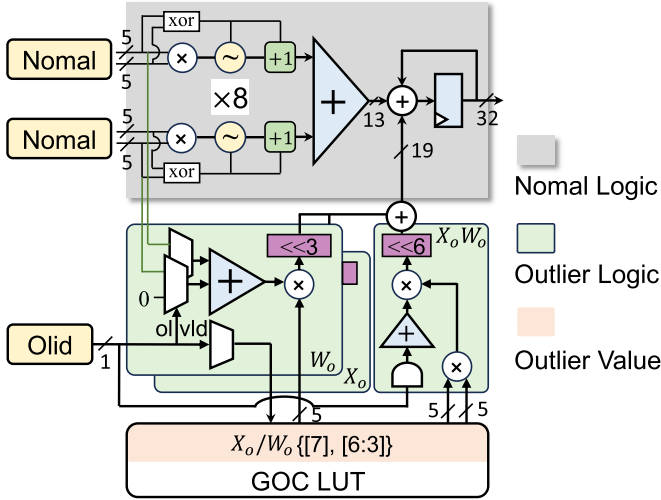


Fig. 10. The structure of outlier-normal mixed PE, composed of normal MAC logic and outlier logic. The normal related logic is represented in the gray block, the outlier logic is represented in the green block. Outlier computation is processed in an interleaved manner.

In addition, we devise an OFQ encoder capable of quickly encoding original activations into low-bit OFQ encoded output and GOC-based outliers. The encoder consists of lightweight hardware components, including comparators, accumulators, shifters, and a multiplexer. As depicted in Fig. 9, the inputs are first compared with q_{max} and q_{min} used in RBC. We allocate two outlier clusters: one for values greater than q_{max} and another for values less than q_{min} , assigning them the corresponding left encodings of -7 and -0 , respectively. If the input is greater than q_{max} , it is accumulated and further shifted by g . Since the group size is always 2^8 , a lightweight shifter is used to calculate the mean for the GOC-based outlier in a specific group. The same process is utilized to calculate outlier for input less than q_{min} . After encoding an entire group of inputs, we generate the specific low-bit value for each input and the GOC-based outliers used for this group.

C. Outlier-Normal Mixed PE

The outlier-normal mixed PE is devised to process the normal part data and outlier part data simultaneously as depicted in Fig. 10. For normal data in the sign-magnitude format, they are multiplied, then converted into 2's complement and accumulated in normal logic. For outlier data, the computation is split into normal MAC (Multiply Accumulate) and outlier

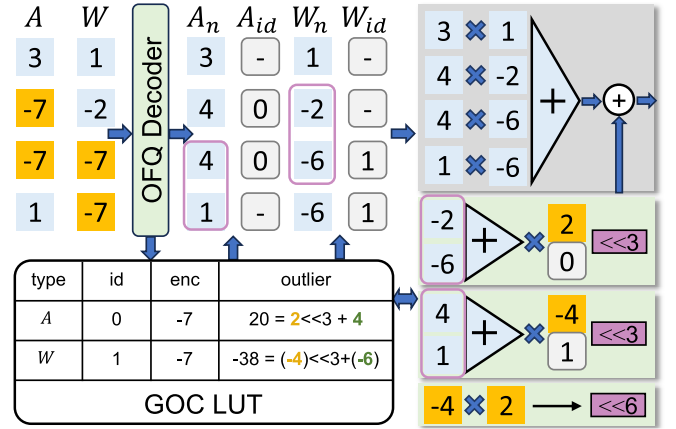


Fig. 11. Dataflow example for outlier-normal mixed PE. In this case, 4 activations and 4 weights with 2 outliers are multiplied and accumulated. Decoders convert OFQ encoding values into oldid and LSP part for outliers, which is 4 for activations and -6 for weights. The MSP part for activation outlier 2 is multiplied by the sum of related weights -2 and -6 . The MSP part for weight outlier -4 is multiplied by the sum of related activations 4 and 1.

MAC which can be formally expressed as:

$$XW = \sum (X^o W^o \ll 6 + (X^o W^n + W^o X^n) \ll 3 + X^n W^n), \quad (15)$$

where X^o and W^o represent the outlier parts, and X^n and W^n denote the normal parts from the decoder. We further split the data into the outlier and the normal part, described as follows:

$$XW = \underbrace{\sum (X^o W^o \ll 6 + (X^o W^n + W^o X^n) \ll 3)}_{\text{outlier}} + \underbrace{\sum X^n W^n}_{\text{normal}}. \quad (16)$$

Since the outliers are clustered into few values and stored in GOC, the outlier computation can be calculated in an interleaved manner:

$$\begin{aligned} \sum X^o W^n &= \sum_{i \in \text{GOC}} X_i^o \left(\sum_j W_j^n \right) \ll 3, \\ \sum W^o X^n &= \sum_{i \in \text{GOC}} W_i^o \left(\sum_j X_j^n \right) \ll 3, \end{aligned} \quad (17)$$

where X_i^o denotes the i_{th} outlier in the GOC LUT, and W_j^n represents the normal values being multiplied. As illustrated in Fig. 10, the outlier computation follows the GOC outlier value sequence. This involves first accumulating all the normal values to be multiplied with a specific GOC outlier, and then multiplying the sum with the outlier part of the corresponding outlier. In addition, $X^o W^o$ is multiplied by the accumulated numbers of the position where both activations and weights are outliers. Finally, these outlier results are shifted and accumulated to generate the total result.

We use the example in Fig. 11 to illustrate the specific dataflow used in our outlier-normal mixed PE. Both activations and weights are input to OFQ decoders and generate the normal value and Olid. The outlier values are split into normal LSP and outlier MSP, where the LSP are decoded into

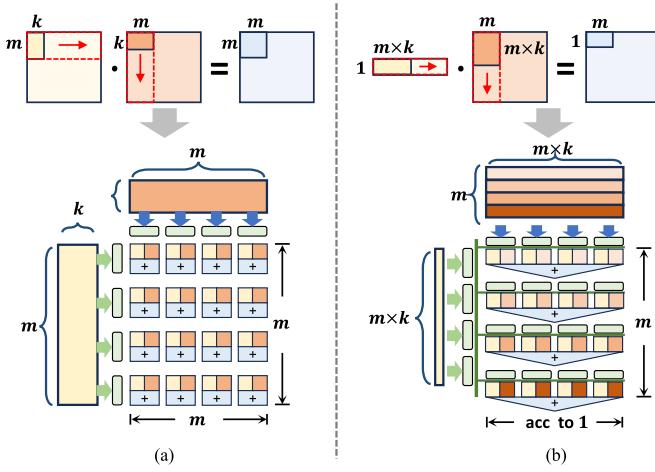


Fig. 12. (a) GEMM dataflow used in the prefilling phase. An $m \times k$ activation matrix is multiplied with a $k \times m$ weight matrix and producing an $m \times m$ partial sum in 1 cycle. Each k activations are broadcasted horizontally and each k weights are broadcasted vertically. (b) GEMV dataflow used in the decoding phase. An $1 \times mk$ activation vector is multiplied with a $mk \times m$ weight matrix and producing an $1 \times m$ partial sum in 1 cycle. The activation vector is broadcasted vertically. Each row of PEs receive different columns of weight matrix and the partial sums are accumulated in row-directions in the reconfigurable accumulator.

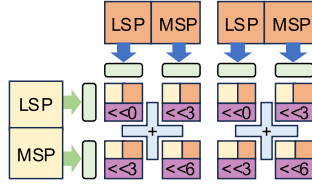


Fig. 13. Mixed-precision dataflow used in the int8-based precision for activations. The 8-bit inputs are divided into MSPs and LSPs and fed into 2 adjacent PE columns/rows. 2×2 adjacent PEs in 2 columns and 2 rows are used to process 8-bit MAC operations and the partial sums are shifted and accumulated in the reconfigurable accumulator.

normal values ensuring that normal computations consistently remain dense workloads. After decoding, the normal values are processed in normal MAC logic while the outlier computations are processed in an outlier interleaved manner. As a result, the outlier-normal mixed PE can process one Olid for both activations and weights per cycle with reduced numbers of multipliers. It is noteworthy that the computations for the outliers are sparse and occur only when outliers are present indicating low efficiency in traditional hardware. In contrast, we only occupy 3 multipliers in our outlier MAC logic and these outlier logic can be effectively gated when there is no outliers.

D. Reconfigurable Dataflow

As mentioned in Section II, LLM inference consists of a prefilling phase and a decoding phase, which utilize GEMM and GEMV operations, respectively. Our OFQ-LLM-based architecture can be configured to support both GEMM and GEMV, effectively accelerating LLM inference. As shown in Fig.12(a), the activation tile and weight tile are processed using an output-stationary dataflow. Here, we assume the PE array size is $m \times m$ and the number of input lanes in each PE is k .

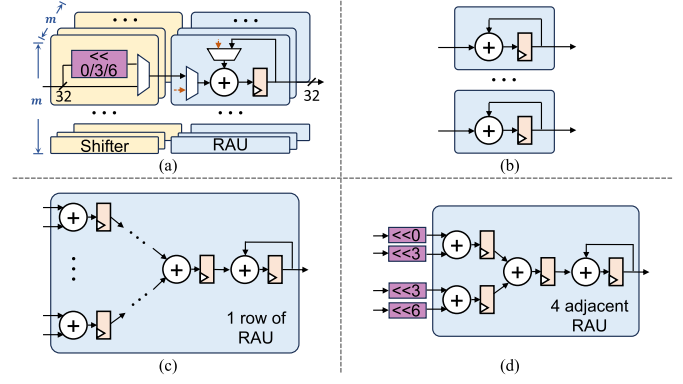


Fig. 14. (a) The hardware structure of reconfigurable accumulator, which consists of multiple shifters and reconfigurable accumulating unit (RAU). The multiplexer in RAU can be configured to receive the data from input or other RAU registers. (b) In GEMM mode, each RAU is configured to an individual accumulator. (c) In GEMV mode, 1 row of RAUs are configured to a row-wise adder tree and an accumulator where RAU registers are used as pipeline registers in adder tree. For m element in a row, $m-1$ RAUs are used to construct adder tree and 1 RAU is used to accumulate final results. (d) In mixed-precision mode, 4 adjacent RAUs are used to accumulate partial sums after specific shifting and the final results are accumulated in the last RAU.

In the GEMM dataflow, $m \times k$ activations are broadcast in the row direction, and $k \times m$ weights are broadcast in the column direction every cycle. Inside each PE, the k activations and k weights are multiplied and accumulated in the inner accumulation register. Once matrix multiplication for the entire tile is completed, the results in each PE are fetched and transferred to the reconfigurable accumulator. In the GEMM dataflow, the reconfigurable accumulating unit (RAU) in reconfigurable accumulator is configured as individual accumulators corresponding to each PE output, shown in Fig.14(b).

The GEMV dataflow used in the decoding phase for LLMs is depicted in Fig.12(b). Here, the activation is a one-dimensional vector with a length of $m \times k$, while the weight is a matrix of size $mk \times m$. The activation vector first passes through the OFQ decoders and is broadcast in the column direction. In contrast, no broadcast is performed for the weight as the size of the weight matrix can fit within the PE array. Therefore, we place one OFQ decoder with each PE for weight decoding. Each row of the PE array computes partial sums corresponding to the results of vector-vector dot products. During the computation of the entire vector and weight tile, the partial sums are initially accumulated within each PE and then added together to obtain the final results. In the GEMV case, the reconfigurable accumulator is transformed into multiple row-wise adder trees to sum all results from one row of the PE array as shown in Fig.14(c).

In our OFQ quantization scheme, we also support int8-based precision for activations in the linear layer following LayerNorm [14]. To accommodate this, we propose a mixed-precision dataflow, as depicted in Fig.12(c), to process 8-bit MAC operations on our PE array. Specifically, we utilize 4 PEs to execute 8-bit MAC operations and deploy shifters in our reconfigurable accumulator to shift partial sums, as shown in Fig.14(c). The four shifted partial sums are then further accumulated in our reconfigurable accumulator. Note that the mixed-precision dataflow is fully compatible with the

TABLE I
PERPLEXITY($\downarrow$) COMPARISON FOR OFQ-LLM AND SOTA QUANTIZATION SCHEMES ON OPT SERIES

Model		OPT-1.3B			OPT-6.7B			OPT-13B		
Bits	Method	WIKI2	PTB	C4	WIKI2	PTB	C4	WIKI2	PTB	C4
FP16	-	14.63	16.96	14.72	10.86	13.09	11.74	10.13	12.34	11.20
W8A8	OliVe [21]	29.26	78.45	17.63	21.31	61.24	13.74	20.95	58.69	12.25
	SPARK [29]	10386	7570	6972	15556	9994	11520	11699	6607	6918
W4A8	SmoothQuant [13]	17.37	21.27	17.43	12.22	15.08	13.36	11.28	14.48	12.19
	OFQ-LLM-S(Ours)	15.50	18.59	15.88	11.32	13.81	12.31	10.82	14.16	11.70
	RPTQ [14]	15.39	17.79	15.48	11.21	13.74	12.11	10.90	13.40	11.62
	QHACoD [20]	15.39	17.75	15.47	11.34	13.51	12.21	10.97	12.83	11.67
	OFQ-LLM-R(Ours)	15.37	17.82	15.44	11.17	13.78	12.09	10.89	13.95	11.62
W4A4	OliVe [21]	69.07	139.87	42.53	46.00	117.82	35.48	55.9	96.17	167.96
	SmoothQuant [13]	22.66	27.53	21.73	14.71	18.42	15.34	14.50	18.63	14.81
	OFQ-LLM-S(Ours)	19.20	22.33	18.32	12.15	14.91	13.00	11.63	14.27	12.20
	RPTQ [14]	16.88	19.23	16.55	12.00	15.17	12.85	12.74	15.76	14.71
	QHACoD [20]	18.31	21.07	17.65	12.09	14.64	13.04	12.62	14.78	13.91
	OFQ-LLM-R(Ours)	16.49	18.98	16.32	11.82	14.25	12.66	12.20	14.22	13.11

proposed GEMM and GEMV dataflows, requiring only additional lightweight shifters and accumulation within adjacent rows. This compatibility allows us to accelerate LLM inference with high PE utilization and efficiency in both the prefilling and decoding phases.

V. EVALUATION

In this section, we evaluate the OFQ-LLM quantization accuracy with various LLMs. We also demonstrate and compare OFQ-LLM-based accelerator's area, performance, core energy efficiency and energy consumption with the state-of-the-art (SOTA) accelerators.

A. Experiment Setup

1) *Models and Datasets*: We evaluate the effectiveness of our quantization scheme across a range of LLMs. Specifically, we assess the perplexity of the OPT series [7], including OPT-1.3B, OPT-6.7B, and OPT-13B, as well as LLaMA-7B [30] and Falcon-7B [31]. We evaluate the perplexity (PPL) of each model for language modeling tasks on the Wikitext2 (WIKI) [32], C4 [33], and Penn Treebank (PTB) [34] datasets. Our quantization scheme is implemented in PyTorch, based on the open-source code of RPTQ [14] which support OPT-series quantization. To further quantize other LLMs, we implement our OFQ-LLM based on SmoothQuant [13]. We refer to these two versions of our quantization as OFQ-LLM-R and OFQ-LLM-S, respectively. The group size of GOC is set to 128 in our evaluation.

2) *Evaluation Baselines*: We first compare OFQ-LLM with the SOTA quantization works, including RPTQ [14], SmoothQuant [13], QHACoD [20], OliVe [21] and SPARK [29]. RPTQ and SmoothQuant are SOTA quantization works for LLMs, while OliVe and SPARK are works using outlier-aware encoding. At the hardware level, We compare the core energy consumption, performance and energy reduction of OFQ-LLM-based accelerators against OliVe [21],

SPARK [29], QHACoD [20] and ANT [35]. To ensure a fair comparison, the size of the on-chip SRAM in all accelerators is set to be the same. Therefore, we focus on the core energy efficiency of all on-chip logic, excluding SRAMs, which provides a clear indication of the efficiency of different designs.

3) *Hardware Implementation*: We implement the OFQ-LLM-based hardware architectures in SystemVerilog HDL and synthesize them using Synopsys Design Compiler (DC) with a TSMC 65nm CMOS technology library. The SRAMs are generated by the TSMC 65nm memory compiler. Power consumption is estimated using Synopsys PrimeTimePX by annotating switching activity from LLM inference. We further build a cycle accurate simulator based on OliVe [21] to evaluate all energy including DRAM, on chip Buffers and Core architectures.

B. Evaluation on OFQ-LLM Quantization Scheme

We first evaluate our quantization scheme on the OPT series and compare OFQ-LLM-S and OFQ-LLM-R with SOTA quantization schemes. To ensure a fair comparison, we apply the same configuration of RPTQ in W4A4 to SmoothQuant and our quantization scheme, which maintain the activations after LayerNorm in 8bit. The group size in QHACoD is set to 128 and K is set to 8 refer to [20]. The perplexity results can be found in Table.I. Although SPARK has demonstrated its limited accuracy loss on CNNs and Transformers [29] in the W8A8 configuration, we apply SPARK encoding on SmoothQuant and observe severe accuracy loss for LLMs, even in the W8A8 mode. OliVe supports both W8A8 and W4A8 modes but still introduces noticeable accuracy loss compared to other SOTA quantization schemes. SmoothQuant has shown good accuracy in the W8A8 configuration but suffers significant accuracy loss at low-bit levels like W4A4. Additionally, RPTQ exhibits SOTA accuracy in W4A8 and W4A4 modes due to its channel-wise reorder technique. QHACoD further improves accuracy through outlier channel sorting

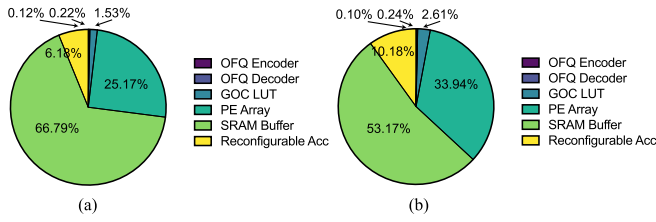


Fig. 15. (a) Area and (b) Power breakdown of OFQ-LLM-based accelerator.

TABLE II
PERPLEXITY(↓) COMPARISON ON LLAMA AND
FLACON WITH SMOOTHQUANT

Model	Bits	Method	WIKI2	PTB	C4
LLaMA 7B	FP16	Baseline	5.68	8.80	7.08
		SmoothQuant	7.01	10.78	8.64
	W4A8	OFQ-LLM-S	6.30	9.65	7.73
		SmoothQuant	19.21	35.15	22.63
Flacon 7B	W4A4	OFQ-LLM-S	11.50	21.19	13.65
		SmoothQuant	19.21	35.15	22.63
	FP16	Baseline	6.59	9.91	9.50
		SmoothQuant	9.04	13.86	12.97
Flacon 40B	W4A8	OFQ-LLM-S	7.15	10.74	10.35
		SmoothQuant	19.21	35.15	22.63
	FP16	Baseline	5.26	7.79	7.86
		SmoothQuant	6.00	9.07	8.88
Flacon 40B	W4A8	OFQ-LLM-S	5.40	8.09	8.01
		SmoothQuant	6.00	9.07	8.88

* W4A8 stands for 4-bit weights and 8-bit activations; W4A4 stands for 4-bit weights and 4-bit activations.

and channel selection. In W4A8 mode, our OFQ-LLM-S can fully outperforms SmoothQuant, OliVe in W8A8, as well as SPARK in W8A8, while exhibiting limited accuracy loss compared to RPTQ and QHACoD. OFQ-LLM-R demonstrate superior accuracy over OliVe, SPARK, SmoothQuant in all datasets. Compared with RPTQ and QHACoD, the W4A8 OFQ-LLM-R achieves better accuracy in Wikitext2 and C4. In W4A4 mode, our OFQ-LLM-S can still achieve better perplexity over OliVe, SPARK and SmoothQuant. Furthermore, our OFQ-LLM-R can exhibit best perplexity in W4A4 mode on all three datasets compared with other works.

Secondly, we compare our quantization scheme on LLaMA and Falcon with SmoothQuant. As shown in Table II, OFQ-LLM-S outperforms SmoothQuant in both W4A8 and W4A4 mode. Especially, in W4A4 mode, our quantization scheme achieves a 7.71 perplexity reduction in Wikitext2. These results demonstrate that the channel-wise scaling used in SmoothQuant is not effective in addressing the outlier quantization issue at low-bit levels. In contrast, OFQ-LLM quantizes the data in both normal and outlier range, providing flexibility for outlier values, which leads to significant perplexity improvements. The perplexity results for Falcon are also depicted in Table II, where we observe similar perplexity reductions across all three datasets, further demonstrating the effectiveness of our quantization scheme.

TABLE III
COMPARISON OF THE PROPOSED CORE ARCHITECTURE
WITH SOTA DESIGNS

Design	ANT [35]	SPARK [29]	OliVe [21]	QHACoD [20]	This Work
Precision (bits)	flint4/int8	int8/4	flint4/int4 /abfloat	int8/4	int8/4
Frequency (MHz)	200	200	200	200	200
Throughput (TOPS)	0.8-1.6	0.8-1.6	2.11	3.2	3.2
Area (mm ²)	4.38	3.95	4.39	4.35	5.32
Power (mW)	496.6	508.2	497.5	611.9	378.5
Energy	1.611	1.574	-	-	-
Efficiency (TOPS/W)	-	-	4.241	5.230	8.454

C. Evaluation on OFQ-LLM Hardware

We implement our OFQ-LLM-based accelerator, ANT, SPARK, OliVe and QHACoD in TSMC 65nm process, operating at 200MHz. Our accelerator contains a 32×32 PE array, with 8 input lanes in each PE, and 640KB on chip SRAM. We set all PE input lanes to 8 in all accelerators. We set the K to 1 in each QHACoD PE. The overall architecture of OFQ-LLM-based accelerator occupies 16.01 mm² and the area and power breakdown are shown in Fig. 15.

1) *Core Architecture Comparison:* Table III shows the detailed comparison of our OFQ-LLM-based accelerator with SOTA designs in TSMC 65nm process. For a fair comparison, we define the “core architecture” as all logic excluding on-chip SRAM, and we assume equivalent SRAM size across all evaluated designs. For consistency, an operation refers to either a multiplication or addition when evaluating throughput. OFQ-LLM utilize outlier-normal mixed PE to support low-bit LLM quantization and computation, our design occupy 5.32 mm² area while the outlier logic occupies 43.57% of total outlier-normal mixed PE and only exhibits 5.44% toggle rate. The outlier logic is separated from the normal logic and only process sparse outlier values, which leads to low power consumption and better throughput. ANT [35] utilizes flint and int8 precision to fit the different distributions in weights and activations. However, the lack of outlier support leads in ANT leads to large data width in both memory access and computation. SPARK [29] utilizes variable length data representation which encode small data in 4-bit which support mixed precision computation, but they do not demonstrate their effectiveness in low bit quantization. In addition, the portion of weights/activations can be compressed into 4-bit is small. OliVe [21] proposes outlier-victim pair quantization and demonstrates its effectiveness in low-bit LLM quantization. However, OliVe requires large decoders to support both normal and outlier value decoding, resulting in large area and power overhead. Furthermore, the data are decoded to exponent and mantissa and processed in an PE with similar logic to floating-point architecture. The complicated decoding and processing of OliVe leads to lower energy efficiency. The core architecture

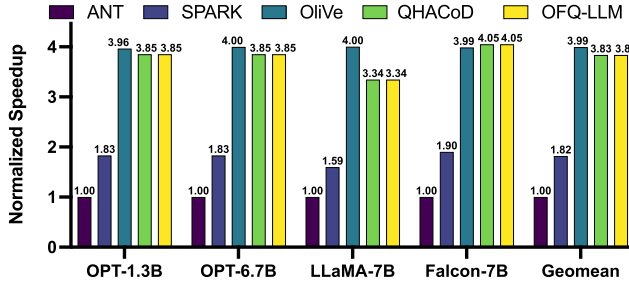


Fig. 16. Performance comparison on LLM prefiling for different accelerators.

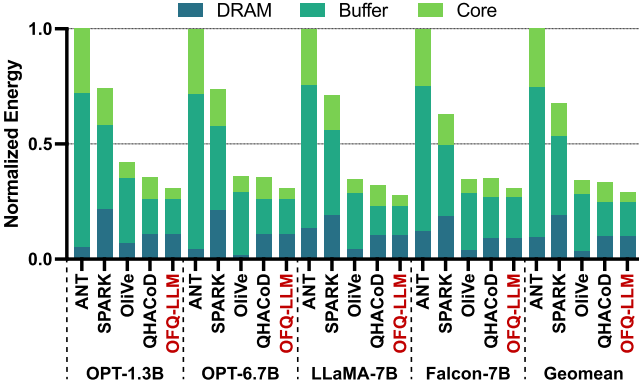


Fig. 17. Comparison of energy on LLM prefiling for different accelerators.

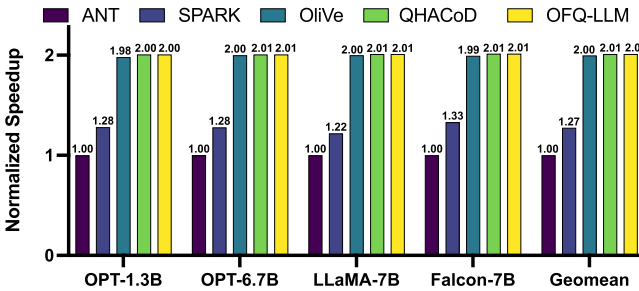


Fig. 18. Performance comparison on LLM decoding for different accelerators.

of OFQ-LLM-based accelerator can achieve $2.62\times$, $2.69\times$, $1.99\times$, $1.62\times$ energy efficiency compared to ANT, SPARK, OliVe and QHACoD. QHACoD leverage additional multiplier and adders in its PE to concurrently handle the outlier channels and support matrix-vector multiplication architecture designated for LLM decoding. Despite this, it achieve 5.230 TOPS/W energy efficiency, which is still lower than ours.

2) *Performance Comparison*: Fig. 16 compares the speedup values of different accelerators on OPT-1.3B, OPT-6.7B, LLaMA-7B and Falcon-7B during the LLM prefill phase and summarize the geometric mean (Geomean) of all benchmarks. ANT [35] lacks the ability to handle outliers, and achieves the lowest performance. SPARK only achieves limited performance improvement over ANT because of limited portion of small data in LLMs which can be encoded and processed at low-bit levels. The speedup value of OliVe is similar to that of QHACoD and OFQ-LLM with a smaller

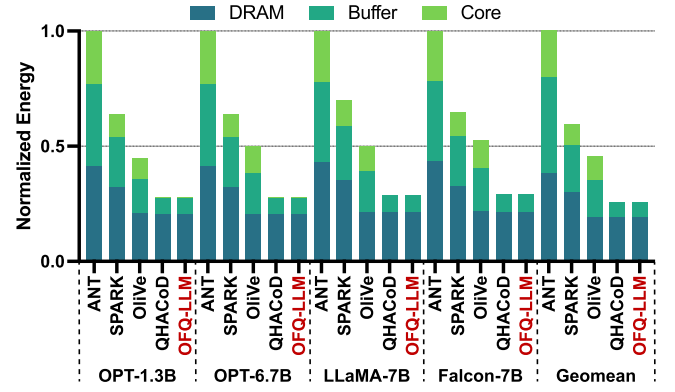


Fig. 19. Comparison of energy on LLM decoding for different accelerators.

throughput. This is because OFQ-LLM and QHACoD used 8-bit activation after two LayerNorm in LLMs, while OliVe encode all activations in 4-bit flint or int4. However, OliVe suffers from significant accuracy loss compared to OFQ-LLM, making it less practical for real-world inference. QHACoD exhibit the same throughput with our design as shown in Table III, which results in similar performance. On average, OFQ-LLM achieves $3.83\times$, $2.10\times$, $0.96\times$, $1.00\times$ speedup over ANT, SPARK, OliVe and QHACoD in LLM prefiling phase. Fig. 18 compares the speedup values of different accelerators on OPT-1.3B, OPT-6.7B, LLaMA-7B and Falcon-7B during the LLM decoding phase. The core computation in decoding phase involves GEMV, and both QHACoD and OFQ-LLM support GEMV architecture, resulting in better performance compared to prefiling phase. However, compared to prefiling phase, the speedup value in the decoding phase are limited to around $2\times$. This limitation reveals the memory-bound nature of the decoding phase, with a large portion of cycles consumed in memory transfers, which are further influenced by the bit-width used in each benchmark. Due to the ability for GEMV computation, OFQ-LLM achieves $2.01\times$, $1.58\times$, $1.01\times$, $1.00\times$ speedup over ANT, SPARK, OliVe and QHACoD in LLM decoding phase.

3) *Energy Comparison*: Fig. 17 and Fig. 19 compare the energy consumption values of different accelerators on OPT-1.3B, OPT-6.7B, LLaMA-7B and Falcon-7B during the LLM prefill phase and decoding phase. Due to the superior energy efficiency and low-bit support, OFQ-LLM achieves significant energy reductions across DRAM, buffer, and core architecture perspectives. Compared with QHACoD, OFQ-LLM achieves $1.01\times$ and $0.76\times$ core energy reduction during the LLM prefill and decoding phases, respectively. Thanks to GEMV support, the core energy consumption of both OFQ-LLM and QHACoD are substantially lower than the DRAM and Buffer energy in decoding phase. Overall, OFQ-LLM achieves $2.44\times$, $1.32\times$, $0.19\times$, $0.15\times$ energy reduction compared to ANT, SPARK, OliVe and QHACoD in LLM prefiling phase, and $2.88\times$, $1.32\times$, $0.77\times$, $0.005\times$ energy reduction compared to ANT, SPARK, OliVe and QHACoD in LLM decoding phase.

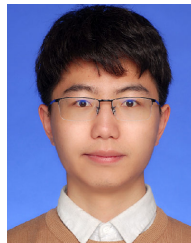
VI. CONCLUSION

This paper proposes OFQ-LLM, an outlier-flexing quantization scheme designed to efficiently accelerate LLMs at low-bit

levels. At the algorithm level, we introduce rescale-based clipping to effectively quantize normal data within a slightly reduced data encoding space. Subsequently, we develop group outlier clustering to efficiently represent flexible outlier values. At the hardware level, an outlier-normal mixed hardware architecture is devised to leverage the encoding scheme and accelerate LLMs with high speed and high energy efficiency. Comparative experiments reveal that our OFQ-LLM achieves better performance compared to SOTA low-bit LLM quantization schemes. In addition, the OFQ-LLM-based accelerator surpasses the SOTA outlier-aware accelerators, achieving up to $2.69\times$ core energy efficiency, up to $3.83\times$ speed up and $2.44\times$ energy reduction in LLM prefilling phase, and up to $2.01\times$ speedup and $2.88\times$ energy reduction in LLM decoding phase, while maintaining superior accuracy.

REFERENCES

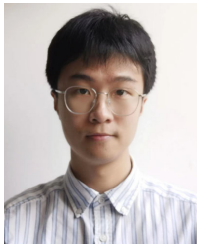
- [1] D. Hendrycks et al., “Measuring massive multitask language understanding,” 2020, *arXiv:2009.03300*.
- [2] S. Li et al., “Evaluating quantized large language models,” 2024, *arXiv:2402.18158*.
- [3] J. Kaplan et al., “Scaling laws for neural language models,” 2020, *arXiv:2001.08361*.
- [4] T. B. Brown et al., “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1877–1901.
- [5] J. Achiam et al., “GPT-4 technical report,” 2023, *arXiv:2303.08774*.
- [6] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” 2023, *arXiv:2307.09288*.
- [7] S. Zhang et al., “OPT: Open pre-trained transformer language models,” 2022, *arXiv:2205.01068*.
- [8] S. Bubeck et al., “Sparks of artificial general intelligence: Early experiments with GPT-4,” 2023, *arXiv:2303.12712*.
- [9] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” 2022, *arXiv:2210.17323*.
- [10] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “GPT3.int8(): 8-bit matrix multiplication for transformers at scale,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 30318–30332.
- [11] S. Kim et al., “SqueezeLLM: Dense-and-Sparse quantization,” 2023, *arXiv:2306.07629*.
- [12] J. Lin et al., “AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration,” in *Proc. Mach. Learn. Syst.*, vol. 6, 2024, pp. 87–100.
- [13] G. Xiao et al., “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *Proc. Int. Conf. Mach. Learn.*, vol. 202, 2023, pp. 38087–38099.
- [14] Z. Yuan et al., “RPTQ: Reorder-based post-training quantization for large language models,” 2023, *arXiv:2304.01089*.
- [15] X. Wei et al., “Outlier suppression: Pushing the limit of low-bit transformer language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 17402–17414.
- [16] J. Liu, R. Gong, X. Wei, Z. Dong, J. Cai, and B. Zhuang, “QLLM: Accurate and efficient low-bitwidth quantization for large language models,” 2023, *arXiv:2310.08041*.
- [17] O. Kovaleva, S. Kulshreshtha, A. Rogers, and A. Rumshisky, “BERT busters: Outlier dimensions that disrupt transformers,” 2021, *arXiv:2105.06990*.
- [18] Y. Zhao, “Atom: Low-bit quantization for efficient and accurate LLM serving,” in *Proc. Mach. Learn. Syst.*, vol. 6, P. Gibbons, G. Pekhimenko, and C. D. Sa, Eds., 2024, pp. 196–209.
- [19] A. Dosovitskiy et al., “An image is worth 16×16 words: Transformers for image recognition at scale,” 2020, *arXiv:2010.11929*.
- [20] W. Li, A. Hu, N. Xu, and G. He, “Quantization and hardware architecture co-design for matrix-vector multiplications of large language models,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 6, pp. 2858–2871, Jun. 2024.
- [21] C. Guo et al., “OliVe: Accelerating large language models via hardware-friendly outlier-victim pair quantization,” in *Proc. 50th Annu. Int. Symp. Comput. Archit.*, Jun. 2023, pp. 1–15.
- [22] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, “GOBO: Quantizing attention-based NLP models for low latency and energy efficient inference,” in *Proc. 53rd Annu. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2020, pp. 811–824.
- [23] E. Park, D. Kim, and S. Yoo, “Energy-efficient neural network accelerator based on outlier-aware low-precision computation,” in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2018, pp. 688–698.
- [24] A. Vaswani et al., “Attention is all you need,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [25] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, Feb. 2024, Art. no. 127063.
- [26] Y. Jiang, L. A. Leiva, P. R. B. Housel, H. R. Tavakoli, J. Kylmälä, and A. Oulasvirta, “UEyes: An eye-tracking dataset across user interface types,” 2024, *arXiv:2402.05202*.
- [27] Z. Li, X. Liu, J. Zhang, and Q. Gu, “RepQuant: Towards accurate post-training quantization of large transformer models via scale reparameterization,” 2024, *arXiv:2402.05628*.
- [28] W. Shao et al., “OmniQuant: Omnidirectionally calibrated quantization for large language models,” 2023, *arXiv:2308.13137*.
- [29] F. Liu et al., “SPARK: Scalable and precision-aware acceleration of neural networks via efficient encoding,” in *Proc. IEEE Int. Symp. High-Performance Comput. Archit. (HPCA)*, Mar. 2024, pp. 1029–1042.
- [30] H. Touvron et al., “LLaMA: Open and efficient foundation language models,” 2023, *arXiv:2302.13971*.
- [31] G. Penedo et al., “The RefinedWeb dataset for falcon LLM: Outperforming curated corpora with web data, and web data only,” 2023, *arXiv:2306.01116*.
- [32] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” 2016, *arXiv:1609.07843*.
- [33] C. Raffel et al., “Exploring the limits of transfer learning with a unified text-to-text transformer,” *J. Mach. Learn. Res.*, vol. 21, pp. 1–67, Jan. 2020.
- [34] M. Marcus et al., “The Penn treebank: Annotating predicate argument structure,” in *Proc. Workshop Hum. Lang. Technol.*, 1994, pp. 114–119.
- [35] C. Guo et al., “ANT: Exploiting adaptive numerical data type for low-bit deep neural network quantization,” in *Proc. 55th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Oct. 2022, pp. 1414–1433.



Gang Wang received the B.S. degree in microelectronics science and engineering and the M.S. degree in integrated circuit engineering from Shanghai Jiao Tong University, Shanghai, China, in 2019 and 2022, respectively, where he is currently pursuing the Ph.D. degree. His current research interests include domain-specific architecture for autonomous driving and AI accelerators.



Siqi Cai is currently pursuing the B.S. degree with the Department of Micro/Nano Electronics, Shanghai Jiao Tong University, China. Her current research interests include image processing and AI accelerators.



Wenjie Li received the B.E. and M.S. degrees from Nanjing University, Nanjing, China, in 2017 and 2020, respectively. He is currently pursuing the Ph.D. degree with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China.

He has published more than ten peer-reviewed articles. His previous research interests included error correction codes and stochastic computing. His current research interests include VLSI design for deep learning.



Guanghui He (Member, IEEE) received the B.S. degree in electronic engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2002, and the Ph.D. degree in electronic engineering from Tsinghua University, Beijing, China, in 2007. He is currently a Professor with the Department of Micro/Nano Electronics and the MoE Key Laboratory of Artificial Intelligence, Shanghai Jiao Tong University. His research interests include energy-efficient algorithms and circuit design for wireless communication, image processing, emerging memory devices, and artificial intelligent systems.



Dongxu Lyu received the B.S. degree in microelectronics science and engineering from Shanghai Jiao Tong University, Shanghai, China, in 2020, where he is currently pursuing the Ph.D. degree with the School of Electronic Information and Electrical Engineering.

His current research interests include the intersection of efficient AI, digital circuits, and systems. The current focus is ASIC design for emerging AI applications, such as 2D/3D vision perception and natural language processing.